

Homework 4: Parsing

Zimo Qi

Question 1

I used the Berkeley Neural Parser and try several sentences. The first three are the sentences in some existing works, while the rest are some ambiguous sentences as well as hard sentences.

Sentences

1. Many years later as he faced the firing squad, Colonel Aureliano Buendía was to remember that distant afternoon when his father took him to discover ice.
 2. At that time Macondo was a village of twenty adobe houses, built on the bank of a river of clear water that ran along a bed of polished stones, which were white and enormous, like prehistoric eggs.
 3. How many years can a mountain exist before it's washed to the sea?
-
4. Time flies like an arrow, fruit flies like a banana.
 5. i think that that that that boy wrote is wrong
 6. The report that the reviewers from the conference that I mentioned disliked was revised
 7. I saw the star with the telescope
 8. I saw the astronomer with the telescope
 9. I saw a astronomer with the telescope

Parsing Structure Style

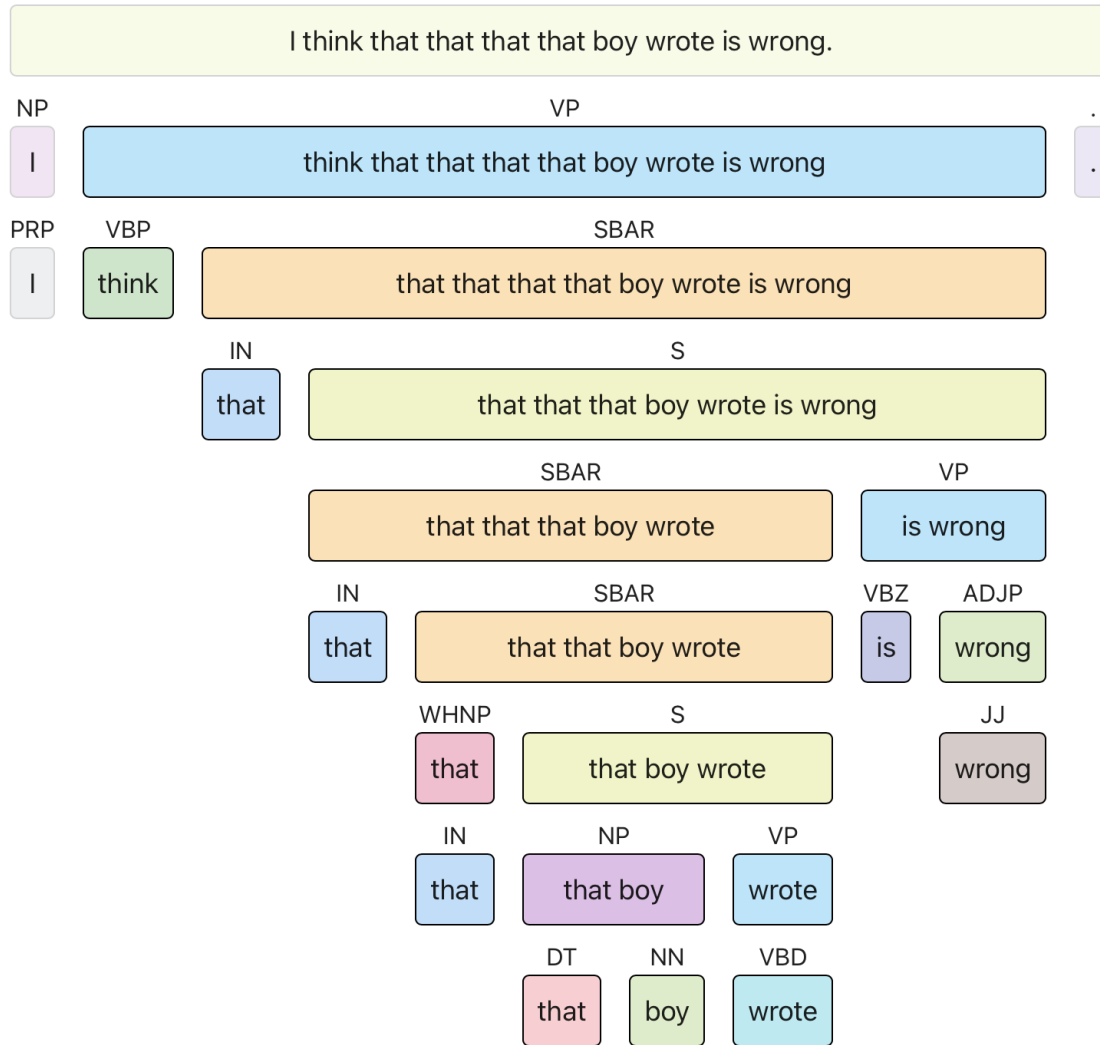
- **The Use of SBAR:** Clause introduced by a (possibly empty) subordinating conjunction); used for complement and adjunct clauses.
- **Morphology:** Verb tense/number and noun number/properness are encoded by some tags (verbs: VB/VBD...; nouns: NN/NNS/NNP/NNPS) rather than node attributes we learned in the class (Verb[tense=xxx,...]).
- **Wh-questions:** Typically analyzed as SBARQ = WH-phrase + SQ. And SQ realizes inverted yes/no question (e.g., MD + NP + VP, VB + NP + NP).
- **Infinitival chains:** Infinitives expand right-branching as VP → TO + VP (→ VB ...), yielding multi-layer VP chains in control/raising contexts. And it the parser wrap infinitive in a bare S (e.g., ... (S (VP (TO to) (VP ...)))) for packaging.

Good and Bad Cases

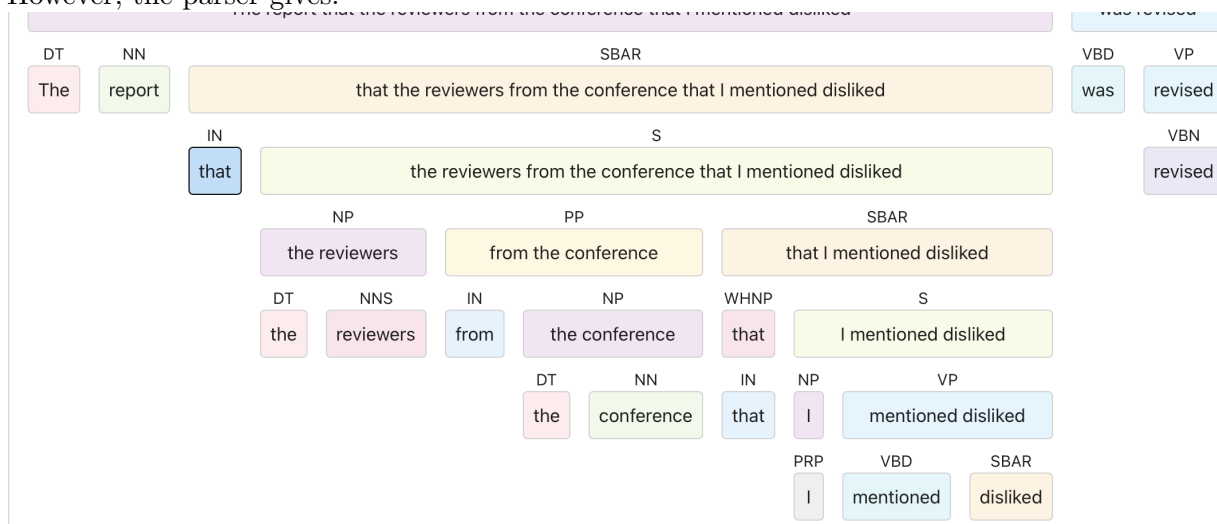
- **(GOOD)** The parser performs reliably on the first three sentences (from the opening of *One Hundred Years of Solitude* and Bob Dylan's *Blowin' in the Wind*). Phrase boundaries (NP/VP/PP) and clause structure (S/SBAR) match well.
- **(GOOD)** For the hard case *I think that that that that boy wrote is wrong*, the parse captures the intended nested structure: outer complement SBAR (IN that) under think, plus an NP head modified

by a relative clause (**that ... wrote**) before the predicate **is wrong**.

S



- **(GOOD)** PP-attachment is handled sensibly in Sentences 7–9. In 7, the PP *with the telescope* attaches to the VP(saw), while in 8–9 it attaches to the NP(*astronomer*).
- **(BAD)** Sentence 6: the analysis mishandles relative-clause scope and argument roles. In the intended parsing, (i) *that I mentioned* modifies *the conference*, and (ii) *the reviewers* are the agents of *disliked*. However, the parser gives:



Ambiguous Failure Cases

- **(BAD)** *Time flies like an arrow; fruit flies like a banana.* The parser prefers a literal “N V” analysis for both clauses (*Time* as NP, *flies* as V), failing to recognize the compound noun *fruit flies* in the second clause. This illustrates a common lexical-ambiguity error absent stronger lexical/semantic cues.

Question 2

(a) Other Parsers

I tried *The report that the reviewers from the conference that I mentioned disliked was revised* for all parsers.

Dependency grammar spaCy:

- The parser regards all Noun Phrase as a whole NOUN.
- The Verbs in the sentence forms some collections of out-degree of dependency arrows.
- The parser **failed** to assign the dependency between *the reviewers* and *disliked* in the example.

Link Grammar Parser

- It correctly parses the sentence, good!

```
Found 2 linkages (2 with no P.P. violations)
Linkage 1, cost vector = (UNUSED=0 DIS=1 AND=0 LEN=44)

      +-----Ss*t-----
      +-----Bs-----
      |               +-----Sp-----
      |               +-----Cr-----+-----Js-----+-----Bs-----
+---Ds---+---R---+   +---Dmc---+---Mp---+   +---Ds---+---R---+---Cr---+
|         |         |         |         |         |         |         |
the report.n that.r the reviewers.n from the conference.n that.r I.p

-----+
-----+
-----+
-----+
Sp*i-+   |   +---Pv---+
|         |   |         |
mentioned.v disliked.v was.v revised.v
```

(b) Other Language: Chinese

I tried the first sentence in *One Hundred Years of Solitude* in Chinese.

- **Dependency grammar spaCy** cannot assign the correct taggings for words. It cannot even distinguish between Nouns and Verbs. For example, it recognize the *ice* in Chinese as a verb.

Question 3

The required new data structures and algorithmic modifications are shown below.

Data Structures

For each column, my parser maintains three core data structures:

- **Agenda:** a queue of unprocessed items. Compared to the simple list used in `recognize.py`, I implemented two queues within the class:
 - a normal queue for newly generated items, as defined before, and
 - a secondary *redo queue* for items whose scores are improved later.

The `pop()` method always prioritizes the redo queue. Thus the updated items are reprocessed first.

- **best[col][item]:** the minimum accumulated inside cost for reaching this item.
- **back[col][item]:** backpointers enabling tree reconstruction. Each operations (PREDICT, SCAN, ATTACH) writes a distinct backpointer tuple.

Also, an auxiliary method `_update_item()` is introduced to unify the logic for all three operations. In the starter code `recognize.py`, the three implements methods respectively.

Operation Logic with Backpointers

Each column is processed by repeatedly popping the best item and calling one of the three operations:

PREDICT: If the next symbol after the dot is a nonterminal, enqueue all its grammar expansions. A backpointer (`'PRED',`) is stored. One may notice the item in the backpointer is omitted since predictions themselves do not form tree nodes.

SCAN: If the next symbol matches the current token, move the dot forward to the next column. A backpointer (`'SCAN', item, token`) records the scanned terminal and the parent item.

ATTACH: Once a constituent is complete, attach it to every waiting customer. A backpointer (`'ATTACH', father_item, child_item`) links the newly completed subtree to its parent.

Bracket Reconstruction

(I think it is actually the most difficult part in this question for me, lol)

I implemented a function `_bracket()` which recursively follows the backpointers to construct a labeled bracket tree.

- For **SCAN**, it appends the corresponding terminal word.
- For **ATTACH**, it recursively builds the constituent for the child item and combines it under the parent's nonterminal.

To be noticed, the recursion produces correct hierarchical structures, even for multi-word lexical rules such as “*chief of staff*”. These rules perplex me a lot in the processing.

Correctness and Efficiency

(a) **Correctness.** Prove the Correctness by Tracking the best derivation and total weight. Every chart column $0, \dots, n$ stores two associative tables:

- **best[col][item]:** the current best accumulated inside cost for reaching the item;
- **back[col][item]:** the backpointer tuple that records how that best derivation was obtained (one of (`'PRED',`), (`'SCAN', prev_item, token`) or (`'ATTACH', parent_item, child_item`)).

All insertions and updates are proceesed by a single helper `_update_item(col,item,new_weight,new_backptr)`. Its logic is:

1. If `item` is not present in `best[col]`, record `best[col][item]=new_weight` and `back[col][item]=new_backptr`, then **insert the item into the column agenda**.
2. If `item` is present but `new_weight < best[col][item]`, replace the stored weight and backpointer and **requeue the item** so it will be reprocessed under the improved cost.
3. Otherwise, ignore it. Because it is not better.

This design ensures *loop invariance*: at any time before the algorithm terminates, `best[col][item]` stores the smallest(best) weight seen so far for that item, and `back[col][item]` points to a concrete derivation achieving that weight. Reconstruction simply follows `back` pointers recursively, so the bracketed tree corresponds exactly to the best derivation recorded in `best`.

When discovering another derivation. As stated before, when a new derivation for an already-seen item is found we compare weights: If it is **worse or equal**, we ignore it. If it is **better**, we update `best` and `back` and requeue the item so its consequences are reconsidered under the improved cost.

(b) Efficiency. We explain why the implementation attains the usual Earley bounds: space $O(n^2)$ and time $O(n^3)$ in the worst case.

Space bound $O(n^2)$. An item is uniquely determined by the triple (rule, dot_position, start_position). Let $|\mathcal{G}|$ be the number of grammar rules and let $L_{\max}$ be the maximum RHS length; both are constants for a fixed grammar. For each start position $s \in \{0, \dots, n\}$ and each rule there are at most $L_{\max} + 1$ distinct dot positions, so the total number of distinct items across all columns is bounded by

$$(|\mathcal{G}|) \cdot (L_{\max} + 1) \cdot (n + 1) = O(n).$$

There are $n + 1$ columns, so the number of stored items across the entire chart is $O(n^2)$. Each stored item have a **constant-sized best** entry and a **constant-sized back pointer**, so the total memory is $O(n^2)$.

Time bound $O(n^3)$. Each item triggers at most one of three constant-time operations: PREDICT, SCAN, or ATTACH. The first two contribute $O(n^2)$ overall because there are only $O(n^2)$ **distinct items across all columns** for a fixed grammar. The **dominant cost arises from ATTACH**: every completed item spanning $[i, j)$ may combine with up to $O(n)$ customers in column i . Multiplying $O(n^2)$ completed items by $O(n)$ possible attachments gives the familiar $O(n^3)$ upper bound. Thus the total runtime for a fixed grammar is $O(n^3)$.

Extra-credit: the reprocessing analysis The above bounds assume the naive case where items are not repeatedly improved. In practice we must handle reprocessing when a better derivation appears in the probabilistic version of Earley's.

- When a lower-cost derivation for an item is found, we simply update its entry in `best` and `back`, and put it back into the agenda to be processed again. To simplify and avoid scanning the whole column repeatedly, the agenda includes a small “redo” queue that gives such reprocessed items higher priority, so they are handled right away instead of waiting behind the existing items.
- **Runtime analysis:** In the given grammar, an item's weight can improve only a limited number of times, say, a constant. Therefore, the total extra work caused by reprocessing adds just a small constant factor to the overall runtime. So in practice the runtime remains close to the ideal cubic bound.

Question 4

My final parser takes advantages of several ideas from Appendix E.

- **E.1** Batch duplicate prediction: avoid re-predicting the same nonterminal in a column
- **E.3** A*-style priority agenda
- **E.5** Left-corner filtering
- **E.7** Vocabulary specialization

E.1: Batch Duplicate Prediction

Naive Earley re-expands a nonterminal every time it appears after the dot, even if it was already predicted in the same column. I keep a column-level set `_predicted_batch[i]` and skip duplicates:

```
1 def _predict(self, nonterminal: str, position: int) -> None:
2     if nonterminal in self._predicted_batch[position]:
3         return
4     self._predicted_batch[position].add(nonterminal)
5     ...
```

This removes many redundant expansions for recursive categories.

E.3: Priority Agenda and A*

I use a *priority queue per column*. The outer loop still processes columns left-to-right, but within each column items are popped by priority.

Cost and Priority. Each item is prioritized by

$$p(i) = g(i) + h(i),$$

where $g(i)$ is the accumulated inside cost and $h(i)$ is an heuristic lower bound on the remaining cost. I use a lightweight heuristic based on the minimum rule weight:

$$h(i) = c_{\min} \times |\text{RHS}_{\text{remaining}}|,$$

where $c_{\min}$ is the smallest rule weight in the grammar (if $c_{\min} = 0$, this reduces to uniform-cost search). This defers long/expensive tails without compromising optimality.

Implementation. I compute $c_{\min}$ once at initialization and use it when (re)queuing:

```
1 class EarleyChart:
2     def __init__(self, tokens, grammar, progress=False):
3         ...
4         self._cmin = self._compute_cmin()
5         self._run_earley()
6
7     def _update_item(self, col_index, item, new_weight, backptr):
8         best, back = self.best[col_index], self.back[col_index]
9         if (item not in best) or (new_weight < best[item]):
10            best[item] = new_weight
11            back[item] = backptr
12            # A* priority: g + h (h = c_min * remaining RHS length)
13            rem = len(item.rule.rhs) - item.dot_position
14            fscore = new_weight + rem * self._cmin
15            if item in self.cols[col_index]._index:
16                self.cols[col_index].requeue(item, fscore)
17            else:
18                self.cols[col_index].push(item, fscore)
```

I also drain stale heap heads before truthiness checks and pops to avoid the “heap non-empty but no valid item” trap.

E.5: Left-Corner Filtering

Before processing column i , I compute the set of nonterminals that can derive the next word via the left-corner relation.

```
1 w = self.tokens[i]
2 seeds = self._preterms_by_word.get(w)
3 if seeds:
4     allowed_lhs, frontier = set(seeds), list(seeds)
5     while frontier:
6         B = frontier.pop()
7         for A in self._P.get(B, ()):
8             if A not in allowed_lhs:
9                 allowed_lhs.add(A); frontier.append(A)
10    self._leftcorner_allowed = allowed_lhs
11 else:
12    self._leftcorner_allowed = None
```

Subsequent `_predict()` calls prune anything not in `allowed_lhs`:

```
1 def _predict(self, nonterminal: str, position: int) -> None:
2     allowed = getattr(self, "_leftcorner_allowed", None)
3     if (allowed is not None) and (nonterminal not in allowed):
4         return
5     ...
```

E.7: Vocabulary Specialization

I specialize the grammar to the input sentence so that rules whose terminals never occur in the sentence are never considered.

Implementation. First, I build a word→preterminals index and cache a per-sentence vocabulary:

```
1 self._R, self._P, self._preterms_by_word = {}, {}, {}
2 for A in self.grammar.nonterminals():
3     for rule in self.grammar.expansions(A):
4         rhs = rule.rhs
5         if not rhs:
6             continue
7         B = rhs[0]
8         if self.grammar.is_nonterminal(B):
9             self._R.setdefault((A, B), []).append(rule)
10            self._P.setdefault(B, set()).add(A)
11        else:
12            self._preterms_by_word.setdefault(B, set()).add(A)
13 self._token_set = set(self.tokens)
```

Then I cache specialized expansions per LHS and consult them in `_predict`:

```
1 def _build_vocab_specialization(self) -> None:
2     tokset = self._token_set
3     is_nonterm = self.grammar.is_nonterminal
4     self._spec_rules = {}
5     def add_rules_for(A):
6         if A in self._spec_rules: return
7         kept = []
8         for rule in self.grammar.expansions(A):
```

```

9         terms = [s for s in rule.rhs if not is_nonterm(s)]
10         if (not terms) or all(t in tokset for t in terms):
11             kept.append(rule)
12         self._spec_rules[A] = kept
13     if hasattr(self.grammar, "nonterminals"):
14         for A in self.grammar.nonterminals(): add_rules_for(A)
15     self._add_spec_for_lhs = add_rules_for
16
17 def _predict(self, nonterminal: str, position: int) -> None:
18     ...
19     if nonterminal not in getattr(self, "_spec_rules", {}):
20         self._add_spec_for_lhs(nonterminal) # lazy build
21     for rule in self._spec_rules.get(nonterminal, []):
22         new_item = Item(rule, dot_position=0, start_position=position)
23         self._update_item(position, new_item, rule.weight, ('PRED',))

```